ZoremGame MCP Project - Documentación Completa

> Plan de implementación de dos MCPs (Model Context Protocol): uno para Unity y otro para Blender
> Objetivo: Automatizar desarrollo 3D e integración, permitiendo a Claude controlar ambas herramientas
> Timeline: 1-2 semanas de implementación intensive

📋 Índice de Documentación

1. **Visión General** - Qué son los MCPs y por qué los necesitamos
2. **Arquitectura** - Cómo se comunican los MCPs entre sí y con Claude
3. **Etapas Unity MCP** - Plan paso a paso para implementar MCP Unity (4 fases)
4. **Etapas Blender MCP** - Plan paso a paso para implementar MCP Blender (4 fases)
5. **Setup & Configuración** - Cómo configurar todo en tu máquina
6. **Testing** - Estrategia de testing y verificación
7. **Referencia Técnica** - Specs detalladas, APIs, ejemplos de código

---

🎯 Resumen Ejecutivo

MCPs a implementar

MCP Unity

Permite a Claude:

- Crear y modificar scripts C#
- Instanciar y configurar GameObjects
- Gestionar prefabs y ScriptableObjects
- Ejecutar compilación y reportar errores
- Automatizar tareas repetitivas

MCP Blender

Permite a Claude:

- Crear y editar modelos 3D
- Configurar rigging (huesos y animaciones)
- Crear y exportar animaciones
- Exportar automáticamente a FBX
- Batch processing de múltiples assets

Ventajas

```
Sin MCPs:
  Yo escribo script → Claude lo genera
  Yo lo copio manualmente a Unity
```

```
    Yo manualmente configuro componentes
    = Lento, propenso a errores

  Con MCPs:
    Yo: "Crea un PlayerController con movimiento"
    MCP: Crea el script directamente en Assets/
    MCP: Configura componentes automáticamente
    MCP: Compila y reporta errores
    = Rápido, sin fricción, 10x más productivo
```

📅 Timeline de Implementación

Semana 1 (Esta semana)

Día 1-2: Setup inicial

- Instalar Node.js, TypeScript, dependencias
- Configurar proyecto base para servers MCP
- Setup de Steamworks.NET en Unity

Día 2-3: MCP Unity Fase 1

- Crear servidor Node.js básico
- Implementar tool: CreateScript
- Testing con un script simple

Día 4-5: MCP Unity Fase 2

- Implementar tool: CompileProject
- Implementar tool: GetErrors
- Testing de compilación

Día 5-6: MCP Blender Fase 1

- Crear servidor Node.js para Blender
- Implementar tool: CreateModel
- Verificar Blender CLI

Día 6-7: Integración básica

- Registrar ambos MCPs en settings.json
- Testing end-to-end simple
- Documentación de bugs encontrados

🔧 Estructura de Carpetas

```
MCP/
├── docs/                          (Documentación)
```

```
│   ├── README.md                  (Este archivo)
│   ├── 01_Vision.md
│   ├── 02_Arquitectura.md
│   ├── 03_Unity_MCP.md
│   ├── 04_Blender_MCP.md
│   ├── 05_Setup.md
│   ├── 06_Testing.md
│   └── 07_Referencia_Tecnica.md
│
├── servers/                       (Servidores MCP)
│   ├── unity/
│   │   ├── package.json
│   │   ├── tsconfig.json
│   │   ├── src/
│   │   │   ├── index.ts           (Entry point)
│   │   │   ├── tools/
│   │   │   │   ├── scripts.ts     (CreateScript, ModifyScript)
│   │   │   │   ├── gameobjects.ts (CreateGO, ConfigureComponent)
│   │   │   │   ├── prefabs.ts     (ReadPrefab, WritePrefab)
│   │   │   │   └── build.ts       (Compile, Build, GetErrors)
│   │   │   ├── unity-cli.ts       (CLI wrapper para Unity)
│   │   │   └── types.ts           (Interfaces)
│   │   └── build/                 (Compilado JavaScript)
│   │
│   └── blender/
│       ├── package.json
│       ├── tsconfig.json
│       ├── src/
│       │   ├── index.ts           (Entry point)
│       │   ├── tools/
│       │   │   ├── models.ts      (CreateModel, EditModel)
│       │   │   ├── rigging.ts     (AddRig, CreateArmature)
│       │   │   ├── animation.ts   (CreateAnimation, EditAnimation)
│       │   │   ├── export.ts      (ExportFBX, BatchExport)
│       │   │   └── utils.ts       (Python generation)
│       │   ├── blender-cli.ts     (CLI wrapper para Blender)
│       │   └── types.ts           (Interfaces)
│       ├── build/                 (Compilado JavaScript)
│       └── python/
│           ├── create_model.py    (Scripts Python para Blender)
│           ├── add_rig.py
│           ├── create_animation.py
│           ├── export_fbx.py
│           └── utils.py
│
├── scripts/                       (Utilidades)
│   ├── setup.sh                   (Instalación automática)
│   ├── test-unity.sh              (Testing Unity MCP)
│   └── test-blender.sh            (Testing Blender MCP)
│
└── config/
    └── settings-example.json      (Ejemplo de configuración)
```

🚀 Próximos Pasos

1. **Lee primero:** 01_Vision.md - Entiende el concepto
2. **Entiende:** 02_Arquitectura.md - Cómo funciona
3. **Implementa:** 03_Unity_MCP.md y 04_Blender_MCP.md
4. **Configura:** 05_Setup.md - Setup completo
5. **Testa:** 06_Testing.md - Verifica que funciona
6. **Consulta:** 07_Referencia_Tecnica.md - Detalles técnicos

📊 Status de Implementación

```
[ ] Fase 1: Setup y infra
[ ] Fase 2: MCP Unity básico (CreateScript)
[ ] Fase 3: MCP Unity completo (todas las tools)
[ ] Fase 4: MCP Blender básico (CreateModel)
[ ] Fase 5: MCP Blender completo (todas las tools)
[ ] Fase 6: Integración y testing end-to-end
[ ] Fase 7: Documentación final y ejemplos
```

Actualizado: 2026-03-29 **Estado:** En planificación